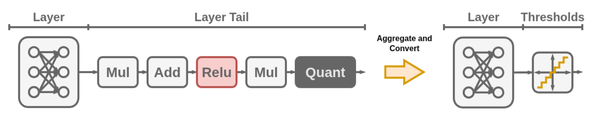
Multi-Thresholds for Non-Monotonic and Fine-Granular Activations

Christoph Berganski, Felix Jentzsch, Marco Platzner
Computer Engineering Group
Paderborn University

SAIL SUSTAINABLE LIFE-CYCLE OF INTELLIGENT SOCIO-TECHNICAL SYSTEMS
Ministerium für Kultur und Wissenschaft des Landes Nordrhein-Westfalen
eki ENERGIEEFFIZIENTE KI DURCH DNN-APPROXIMATION FÜR FPGAs
PADERBORN UNIVERSITY
Supported by: Federal Ministry for the Environment, Nature Conservation, Nuclear Safety and Consumer Protection
based on a decision of the German Bundestag

Layer Tail Compilation in FINN

SIRA: Scaled-Integer Range Analysis for Optimizing FPGA Dataflow Neural Network Accelerators[1]



..., aggressive quantization can expose non-matrix-multiply operations as significant performance and resource bottlenecks...

Multi-Threshold Functions

$$f_{\Theta,\delta}(x) = b_\Theta + s_\Theta \sum_{i=0}^{N-1} \delta_i \cdot (x \geq \Theta_i)$$

Closely related to the **uniform quantizer** **Q** with scale s_Θ, bias b_Θ and bit width β

$$\mathbf{Q}(x) = s_\Theta \cdot \mathrm{clip}\left(\left\lfloor \frac{x - b_\Theta}{s_\Theta} \right\rfloor; 0, 2^\beta - 1\right) + b_\Theta$$

Current Limitations

Monotonicity, per-channel granularity, no systematic compilation

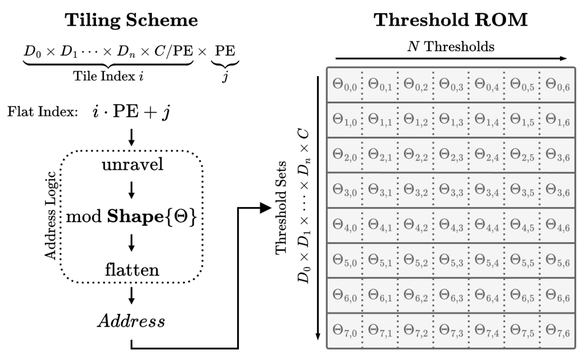
Fine-Granularity

Broadcasting of Thresholds

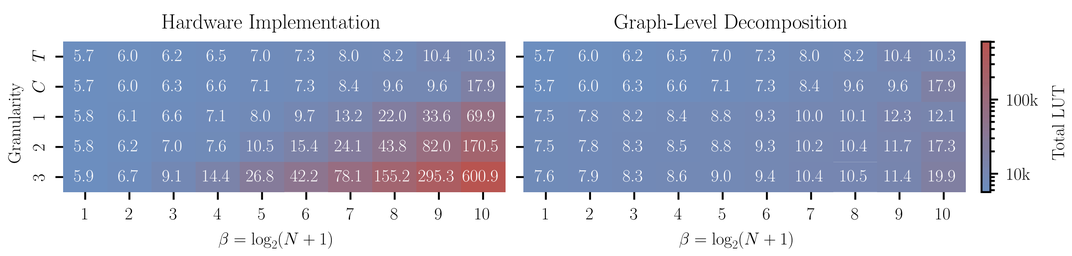
$$\mathbf{Shape}\{\Theta\} \subseteq \mathbf{Shape}\{X\} \times N$$
$$\mathbf{Shape}\{\Theta\} = D_0 \times D_1 \times \cdots \times D_n \times C \times N$$

Graph Level Decomposition

Exact solution possible, if granularity originates from prior absorption of additions (e.g., a bias)



Adress logic and layout of threshold parameter memory.



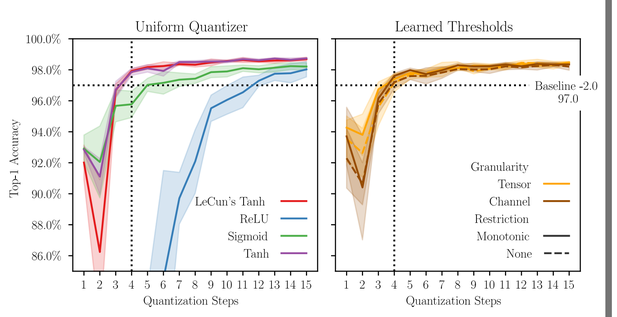
Learning Thresholds

Differentiable Relaxation

$\delta_i = sign(w_i) \approx w_i, |w_i| \approx 1, \ \delta_i \in \{-1, +1\}, \ \alpha > 1$

$$f_{\Theta,\delta}(x) \approx b_\Theta + s_\Theta \sum_{i=0}^{N-1} w_i \cdot \sigma\big(\alpha \cdot (x - \Theta_i)\big)$$

Quantization Aware Training with close correspondence to the target hardware

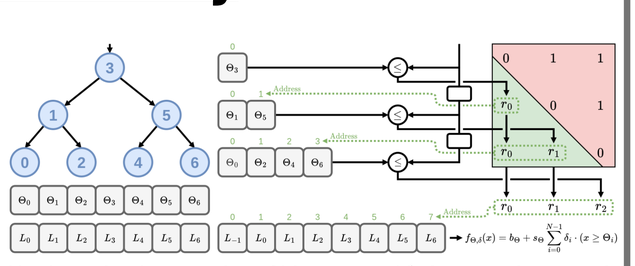


Comparison of learned multi-threshold functions to hand-selected and quantized activation functions for MNIST classification with LeNet.

Non-Monotonicity

Precompute Cumulative Sum of Steps

$$f_{\Theta,\delta}(x) = b_\Theta + s_\Theta L_i \quad \text{s.t. } \Theta_i \leq x < \Theta_{i+1}$$
$$L_i = \sum_{j=0}^{i} \delta_i\,, \quad L_{-1} = 0$$



Hardware implementation of binary search[1]: A remapping stage is added to allow for non-monotonic output levels L_i.

Graph Level Decomposition

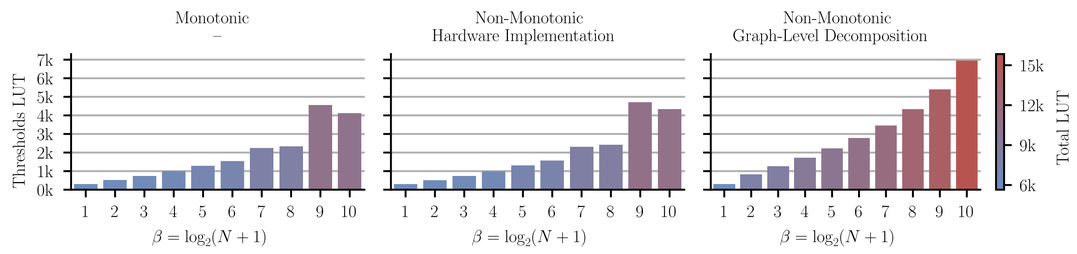
Works for $\delta_i = \pm 1$ and does not require new hardware

Two Bit Widths

Threshold Bit Width $\beta = \log_2(N + 1)$

Quantization Bit Width $\beta^* = \log_2(\max L_i - \min L_i + 1)$

For Monotonic Functions $\beta = \beta^*$



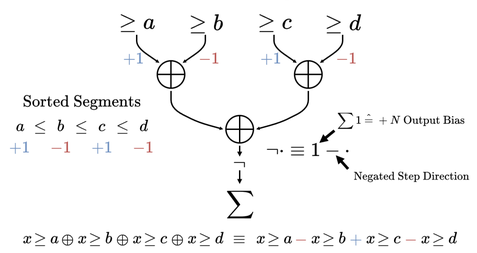
Compiling Functions to Thresholds

Absorption via Inverse Functions

Solve $f(x) \geq \Theta_i$ for x via a suitable inverse function.

For non-monotonic functions needs to consider all branches, e.g.,
$x^2 \geq a \Leftrightarrow x \geq +\sqrt{a} \vee x \leq -\sqrt{a}$

Applied repeatedly to absorb consecutive functions: **branching threshold segments**

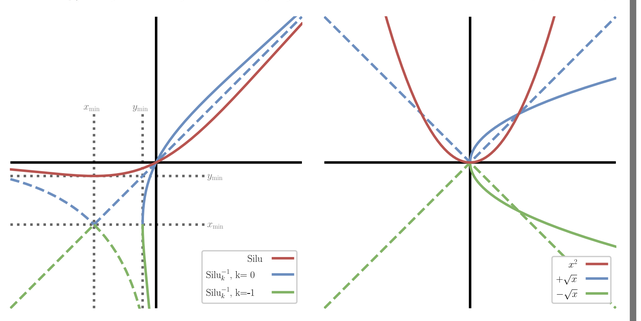


$$x \geq a \oplus x \geq b \oplus x \geq c \oplus x \geq d \ \equiv \ x \geq a - x \geq b + x \geq c - x \geq d$$

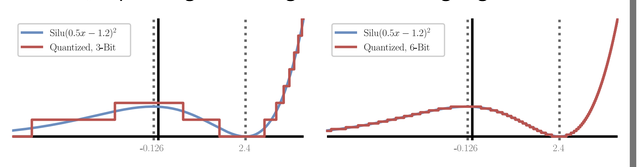
Example: Silu Activation Function

$\mathrm{Silu}(x) = x \cdot \sigma(x)$

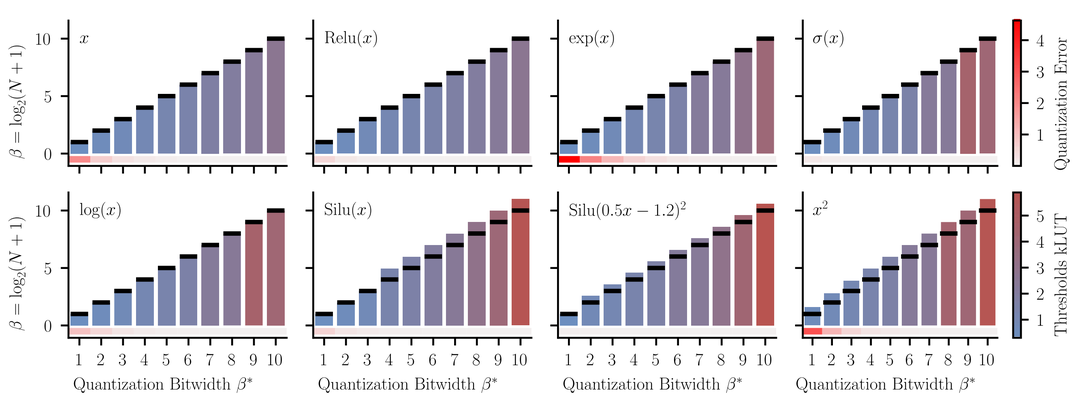
$\mathrm{Silu}_k^{-1}(y) = y + W_k(ye^{-y})$, where $k \in \{0, -1\}$



Inverse of non-monotonic functions: Branch points are at the extrema, separating increasing from decreasing segments.



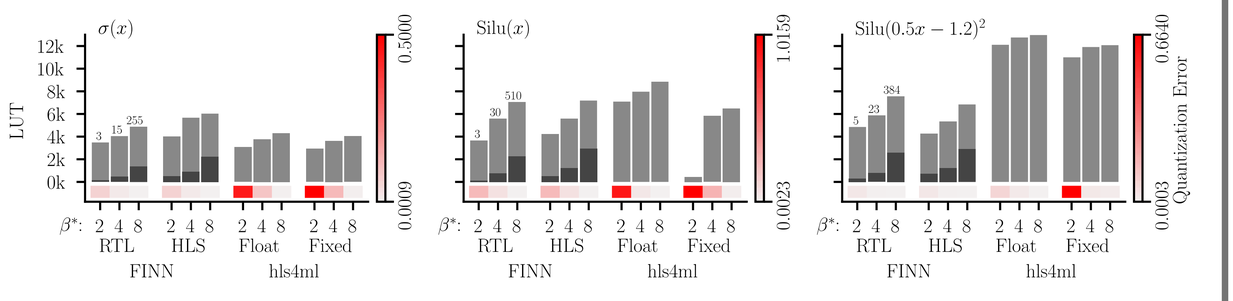
Non-monotonic test function composition with two increasing segments and one decreasing segment.



Evolution of Thresholding in FINN

	Original[2,3]	SIRA[1]	Ours
Compilation			
Approach	Manual *Fixed Patterns*	Automatic *Search-Based*	Semi-Automatic *Fusing Primitives*
Effort	Low	High*	Low
Support			
Monotonicity	Monotonic	Monotonic	Monotonic Non-Monotonic
Granularity	Per-Channel	Per-Channel	Arbitrary
Backend			
Algorithm	Summation	Binary Search	Binary Search
Implementation	HLS	RTL	HLS/RTL*
Extensibility	Add Converter per Pattern	*Not Necessary*	Add Inverse per Primitive

Comparing to Prior Work and *hls4ml*[4]



Comparison of functions implemented as multi-threshold operators in FINN and a composite lookup-table based approach in hls4ml in terms of LUT count over quantization bitwidth β^*. For FINN, HLS refers to our binary-search backend (Section III) and RTL refers to the backend introduced by SIRA, which requires graph-level decomposition for the non-monotonic functions.

References

1. Umuroglu, Y., Berganski, C., Jentzsch, F., Danilowicz, M., Kryjak, T., Bezaitis, C., ... & Blott, M. (2025). SIRA: Scaled-Integer Range Analysis for Optimizing FPGA Dataflow Neural Network Accelerators. *arXiv preprint arXiv:2508.21493*.
2. Umuroglu, Y., Fraser, N. J., Gambardella, G., Blott, M., Leong, P., Jahre, M., & Vissers, K. (2017, February). Finn: A framework for fast, scalable binarized neural network inference. In *Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays* (pp. 65-74).
3. Blott, M., Preußer, T. B., Fraser, N. J., Gambardella, G., O'brien, K., Umuroglu, Y., ... & Vissers, K. (2018). FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, *11*(3), 1-23.
4. FastML Team. (2026). hls4ml (Version v1.3.0) [Computer software]. https://doi.org/10.5281/zenodo.19134628